postgresSQL_lab1

1. Create your tables with their columns in PostgreSQL.
2. Insert at minimum 3 Rows at each table.

```
myproject=# \dt
           List of relations
 Schema |   Name    | Type  |  Owner
--------+-----------+-------+----------
 public | exam      | table | postgres
 public | grades    | table | postgres
 public | stu_sub   | table | postgres
 public | student   | table | postgres
 public | subject   | table | postgres
 public | track     | table | postgres
 public | track_sub | table | postgres
(7 rows)

myproject=#
```

```
myproject=# select * from student;
 id | e_name |      email      | address | track_ic
----+--------+-----------------+---------+---------
  1 | Ahmed  | ahmed@gmail.com | Cairo   |        1
  2 | Sara   | sara@gmail.com  | Assiut  |        2
  3 | Omar   | omar@gmail.com  | Giza    |        1
(3 rows)

myproject=#
```

3. Add birth date column for the student table.

```
myproject=# alter table student add column birth_date DATE;
ALTER TABLE
myproject=# \d student
myproject=# select * from student;
 id | e_name |      email      | address | track_id | birth_date
----+--------+-----------------+---------+----------+------------
  1 | Ahmed  | ahmed@gmail.com | Cairo   |        1 |
  2 | Sara   | sara@gmail.com  | Assiut  |        2 |
  3 | Omar   | omar@gmail.com  | Giza    |        1 |
(3 rows)

myproject=# UPDATE student SET birth_date = '2000-01-15' WHERE id = 1;
UPDATE student SET birth_date = '1990-05-20' WHERE id = 2;
UPDATE student SET birth_date = '1995-08-10' WHERE id = 3;
UPDATE 1
UPDATE 1
UPDATE 1
myproject=#
```

4. Add gender column which hold only 2 values (Male or Female).

```
myproject=# CREATE TYPE gender_type AS ENUM ('Male', 'Female');
CREATE TYPE
myproject=# ALTER TABLE student
ADD COLUMN gender gender_type;
ALTER TABLE
myproject=# UPDATE student SET gender = 'Male' WHERE id = 1;
UPDATE student SET gender = 'Female' WHERE id = 2;
UPDATE student SET gender = 'Male' WHERE id = 3
UPDATE 1
UPDATE 1
myproject-# select * from student;
ERROR:  syntax error at or near "select"
LINE 2: select * from student;
        ^
myproject=# select * from student;
 id | e_name |      email      | address | track_id | birth_date | gender
----+--------+-----------------+---------+----------+------------+--------
  3 | Omar   | omar@gmail.com  | Giza    |        1 | 1995-08-10 |
  1 | Ahmed  | ahmed@gmail.com | Cairo   |        1 | 2000-01-15 | Male
  2 | Sara   | sara@gmail.com  | Assiut  |        2 | 1990-05-20 | Female
(3 rows)

myproject=#
```

5. Add/Alter foreign key constrains in your tables.

```
myproject=# ALTER TABLE grades
ADD CONSTRAINT fk_grades_subject
FOREIGN KEY (sub_id) REFERENCES subject(id);
ALTER TABLE
myproject=# ALTER TABLE grades
ADD CONSTRAINT fk_grades_student
FOREIGN KEY (stu_id) REFERENCES student(id);
ALTER TABLE
myproject=# \d grades
              Table "public.grades"
 Column  |  Type   | Collation | Nullable | Default
---------+---------+-----------+----------+---------
 stu_id  | integer |           | not null |
 sub_id  | integer |           | not null |
 exam_id | integer |           | not null |
 grade   | integer |           |          |
Indexes:
    "grades_pkey" PRIMARY KEY, btree (stu_id, sub_id, exam_id)
Check constraints:
    "grades_grade_check" CHECK (grade >= 0)
Foreign-key constraints:
    "fk_grades_student" FOREIGN KEY (stu_id) REFERENCES student(id)
    "fk_grades_subject" FOREIGN KEY (sub_id) REFERENCES subject(id)
    "grades_exam_id_fkey" FOREIGN KEY (exam_id) REFERENCES exam(id)
```

6. Display male students who are born before 1991-10-01.

```
myproject=# select * from student where gender='Male' and birth_date < '1991-10-
01';
 id | e_name | email | address | track_id | birth_date | gender
----+--------+-------+---------+----------+------------+--------
(0 rows)

myproject=#
```

7. Display students' names that begin with A.

```
myproject=# select e_name from student where e_name LIKE 'A%';
 e_name
--------
 Ahmed
(1 row)

myproject=#
```

8. Display subjects and their max score sorted by max score.

```
SELECT sub.sub_name, MAX(g.grade) AS max_score
FROM grades g
JOIN subject sub ON g.sub_id = sub.id
GROUP BY sub.sub_name
ORDER BY max_score DESC;
       sub_name       | max_score
----------------------+-----------
 web developer        |        90
 python programming   |        85
 Database fundamental |        70
(3 rows)

myproject=#
```

9. Display the subject with highest max score

```
myproject=# SELECT s.e_name, sub.sub_name, g.grade
FROM grades g
JOIN student s ON g.stu_id = s.id
JOIN subject sub ON g.sub_id = sub.id
WHERE g.grade = (SELECT MAX(grade) FROM grades);
 e_name |   sub_name    | grade
--------+---------------+-------
 Sara   | web developer |    90
(1 row)

myproject=#
```

Lab2

1. Display the number of students their name is “Mohammed”

```
myproject=# select sum(id) from student where e_name ='Ahmed';
 sum
-----
   1
(1 row)

myproject=# select count(*) as num_student from student where e_name ='Ahmed';
 num_student
-------------
           1
(1 row)

myproject=#
```

2. Display the number of males and females.

```
myproject=# select gender,count(*) as num_gender from student group by gender;
 gender | num_gender
--------+------------
 Female |          1
 Male   |          2
(2 rows)

myproject=#
```

3. Display the repeated first names and their counts if higher than 2.

```
(2 rows)

myproject=# SELECT e_name, COUNT(*) AS name_count FROM student GROUP BY e_name H
AVING COUNT(*) > 2;
 e_name | name_count
--------+------------
(0 rows)

myproject=#
```

4. Display all students and track name they belong to.

```
myproject=# SELECT student.e_name, track.track_name FROM student JOIN track ON s
tudent.track_id = track.id;
 e_name | track_name
--------+------------
 Ahmed  | python
 Sara   | javascript
 Omar   | database
(3 rows)
```

5. Display all students except those who are in OS track.

```
myproject=# SELECT student.e_name, track.track_name FROM student
JOIN track ON student.track_id = track.id
WHERE track.track_name <> 'python';
 e_name | track_name
--------+------------
 Sara   | javascript
 Omar   | database
(2 rows)
```